

Ex. No. 1 Date: 08/07/2026	CREATING AND MANAGING TABLES	URK24CS1021
Aim (a) Creating and Managing Tables To execute DDL commands and get the desired output. (b) Managing Tables using DML, DCL and TCL Commands To execute all commands of Data Manipulation Language, Data Control Language, and Transaction Control Language to get the desired output. Description (a) Creating and Managing Tables DDL refers to "Data Definition Language", a subset of SQL statements that change the structure of the database schema in some way, typically by creating, deleting, or modifying schema objects such as databases, tables, and views. Most DDL statements start with the keywords CREATE, DROP, or ALTER. (b) Managing Tables using DML, DCL and TCL Commands DML statements (INSERT/UPDATE/DELETE) manipulate row data. DCL statements (GRANT/REVOKE) control access privileges on database objects for specific roles. TCL statements (COMMIT/ROLLBACK/SAVEPOINT) control transaction boundaries — COMMIT makes changes permanent, SAVEPOINT marks a point to roll back to within an open transaction, and ROLLBACK TO undoes work back to that point without discarding the whole transaction. Questions (a) Creating and Managing Tables 1. Create User, Event, Venue, and ticket tables based on the given schema. <pre> CREATE TABLE URK24CS1021_users (userid numeric(10) PRIMARY KEY, name varchar(255), email varchar(255), password varchar(255), phone varchar(20)); CREATE TABLE URK24CS1021_venue (venueid numeric(10) PRIMARY KEY, name varchar(255), address varchar(255), city varchar(255), state varchar(255), country varchar(255)); CREATE TABLE URK24CS1021_event (eventid numeric(10) PRIMARY KEY, name varchar(255), eventdate date, eventtime timestamp, venueid numeric(10), </pre>		

```
        description varchar(500)
    );
CREATE TABLE URK24CS1021_ticket (
    ticketid numeric(10) PRIMARY KEY,
    eventid numeric(10) REFERENCES URK24CS1021_event(eventid),
    userid numeric(10),
    seatnumber varchar(20),
    price numeric(10,2),
    status varchar(50)
);
```

psql — labdb

```
labdb=# CREATE TABLE URK24CS1021_users (  
labdb(#      userid numeric(10) PRIMARY KEY,  
labdb(#      name varchar(255),  
labdb(#      email varchar(255),  
labdb(#      password varchar(255),  
labdb(#      phone varchar(20)  
labdb(# );  
CREATE TABLE  
  
labdb=# CREATE TABLE URK24CS1021_venue (  
labdb(#      venueid numeric(10) PRIMARY KEY,  
labdb(#      name varchar(255),  
labdb(#      address varchar(255),  
labdb(#      city varchar(255),  
labdb(#      state varchar(255),  
labdb(#      country varchar(255)  
labdb(# );  
CREATE TABLE  
  
labdb=# CREATE TABLE URK24CS1021_event (  
labdb(#      eventid numeric(10) PRIMARY KEY,  
labdb(#      name varchar(255),  
labdb(#      eventdate date,  
labdb(#      eventtime timestamp,  
labdb(#      venueid numeric(10),  
labdb(#      description varchar(500)  
labdb(# );  
CREATE TABLE  
  
labdb=# CREATE TABLE URK24CS1021_ticket (  
labdb(#      ticketid numeric(10) PRIMARY KEY,  
labdb(#      eventid numeric(10) REFERENCES URK24CS1021_event(eventid),  
labdb(#      userid numeric(10),  
labdb(#      seatnumber varchar(20),  
labdb(#      price numeric(10,2),  
labdb(#      status varchar(50)  
labdb(# );  
CREATE TABLE  
  
labdb=#
```

2. Describe the tables.

```
\d URK24CS1021_users  
\d URK24CS1021_event  
\d URK24CS1021_venue  
\d URK24CS1021_ticket
```

```
psql — labdb

labdb=# \d URK24CS1021_users
               Table "public.urk24cs1021_users"
  Column |          Type          | Collation | Nullable | Default
-----+-----+-----+-----+-----
userid  | numeric(10,0)          |           | not null |
name    | character varying(255) |           |          |
email   | character varying(255) |           |          |
password| character varying(255) |           |          |
phone   | character varying(20)  |           |          |
Indexes:
    "urk24cs1021_users_pkey" PRIMARY KEY, btree (userid)

labdb=# \d URK24CS1021_event
               Table "public.urk24cs1021_event"
  Column |          Type          | Collation | Nullable | Default
-----+-----+-----+-----+-----
eventid | numeric(10,0)          |           | not null |
name    | character varying(255) |           |          |
eventdate | date                  |           |          |
eventtime | timestamp without time zone |           |          |
venueid  | numeric(10,0)          |           |          |
description | character varying(500) |           |          |
Indexes:
    "urk24cs1021_event_pkey" PRIMARY KEY, btree (eventid)
Referenced by:
    TABLE "urk24cs1021_ticket" CONSTRAINT "urk24cs1021_ticket_eventid_fkey" FOREIGN KEY (eventid) REFERENCES urk24cs1021_event(eventid)

labdb=# \d URK24CS1021_venue
               Table "public.urk24cs1021_venue"
  Column |          Type          | Collation | Nullable | Default
-----+-----+-----+-----+-----
venueid | numeric(10,0)          |           | not null |
name    | character varying(255) |           |          |
address | character varying(255) |           |          |
city    | character varying(255) |           |          |
state   | character varying(255) |           |          |
country | character varying(255) |           |          |
Indexes:
    "urk24cs1021_venue_pkey" PRIMARY KEY, btree (venueid)

labdb=# \d URK24CS1021_ticket
               Table "public.urk24cs1021_ticket"
  Column |          Type          | Collation | Nullable | Default
-----+-----+-----+-----+-----
ticketid | numeric(10,0)          |           | not null |
eventid  | numeric(10,0)          |           |          |
userid   | numeric(10,0)          |           |          |
seatnumber | character varying(20) |           |          |
price    | numeric(10,2)          |           |          |
status   | character varying(50)  |           |          |
Indexes:
    "urk24cs1021_ticket_pkey" PRIMARY KEY, btree (ticketid)
Foreign-key constraints:
    "urk24cs1021_ticket_eventid_fkey" FOREIGN KEY (eventid) REFERENCES urk24cs1021_event(eventid)

labdb=#
```

3. Alter the User table to add a new column Age.

```
ALTER TABLE URK24CS1021_users ADD COLUMN age numeric(3);
```

```
psql — labdb

labdb=# ALTER TABLE URK24CS1021_users ADD COLUMN age numeric(3);
ALTER TABLE

labdb=#
```

4. Drop the newly added column Age.

```
ALTER TABLE URK24CS1021_users DROP COLUMN age;
```

psql — labdb

```
labdb=# ALTER TABLE URK24CS1021_users DROP COLUMN age;  
ALTER TABLE
```

labdb=#

5. Rename the Venue table to Location.

```
ALTER TABLE URK24CS1021_venue RENAME TO URK24CS1021_location;
```

psql — labdb

```
labdb=# ALTER TABLE URK24CS1021_venue RENAME TO URK24CS1021_location;  
ALTER TABLE
```

labdb=#

6. Modify the size of the Event table's Description column to 1000.

```
ALTER TABLE URK24CS1021_event ALTER COLUMN description TYPE varchar(1000);
```

psql — labdb

```
labdb=# ALTER TABLE URK24CS1021_event ALTER COLUMN description TYPE varchar(1000);  
ALTER TABLE
```

labdb=#

7. Drop the SeatNumber column from the Ticket table.

```
ALTER TABLE URK24CS1021_ticket DROP COLUMN seatnumber;
```

psql — labdb

```
labdb=# ALTER TABLE URK24CS1021_ticket DROP COLUMN seatnumber;  
ALTER TABLE
```

labdb=#

8. Add a unique constraint on the Email column in the User table.

```
ALTER TABLE URK24CS1021_users ADD CONSTRAINT users_email_uq UNIQUE (email);
```

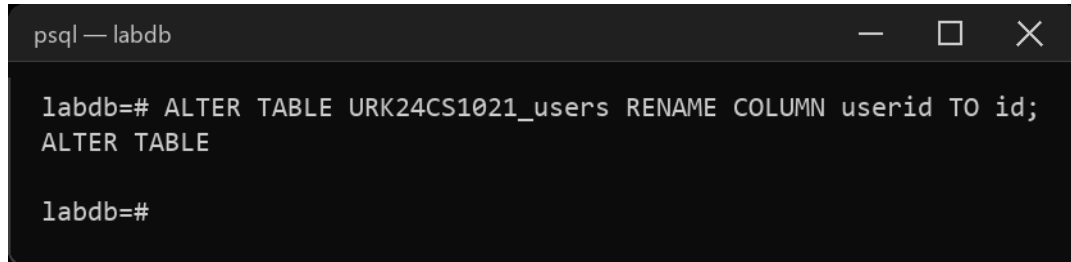
psql — labdb

```
labdb=# ALTER TABLE URK24CS1021_users ADD CONSTRAINT users_email_uq UNIQUE (email);  
ALTER TABLE
```

labdb=#

9. Rename the UserID column in the User table to ID.

```
ALTER TABLE URK24CS1021_users RENAME COLUMN userid TO id;
```

A terminal window titled 'psql — labdb' with standard window controls. It shows the execution of the SQL command 'ALTER TABLE URK24CS1021_users RENAME COLUMN userid TO id;'. The prompt 'labdb=#' is shown before and after the command, with the command itself appearing on the line between the prompts.

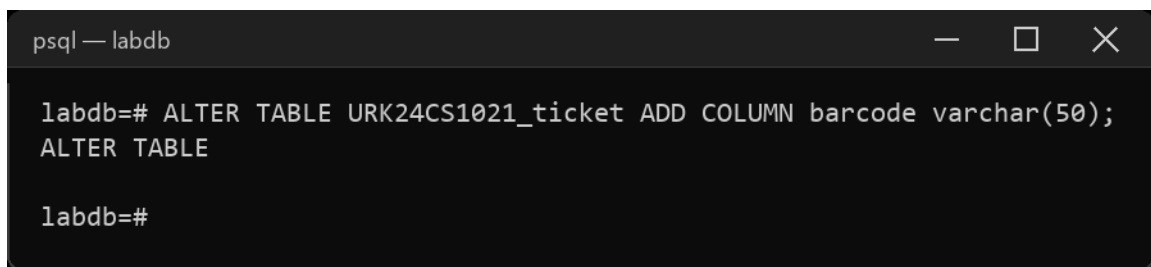
```
psql — labdb

labdb=# ALTER TABLE URK24CS1021_users RENAME COLUMN userid TO id;
ALTER TABLE

labdb=#
```

10. Modify the Ticket table to add a column named "Barcode" with a data type of VARCHAR(50).

```
ALTER TABLE URK24CS1021_ticket ADD COLUMN barcode varchar(50);
```

A terminal window titled 'psql — labdb' with standard window controls. It shows the execution of the SQL command 'ALTER TABLE URK24CS1021_ticket ADD COLUMN barcode varchar(50);'. The prompt 'labdb=#' is shown before and after the command, with the command itself appearing on the line between the prompts.

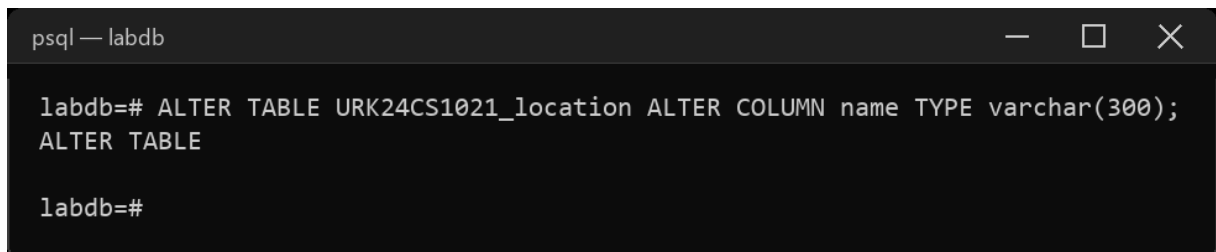
```
psql — labdb

labdb=# ALTER TABLE URK24CS1021_ticket ADD COLUMN barcode varchar(50);
ALTER TABLE

labdb=#
```

11. Modify the Name column in the Venue table to increase its maximum length to VARCHAR(300).

```
ALTER TABLE URK24CS1021_location ALTER COLUMN name TYPE varchar(300);
```

A terminal window titled 'psql — labdb' with standard window controls. It shows the execution of the SQL command 'ALTER TABLE URK24CS1021_location ALTER COLUMN name TYPE varchar(300);'. The prompt 'labdb=#' is shown before and after the command, with the command itself appearing on the line between the prompts.

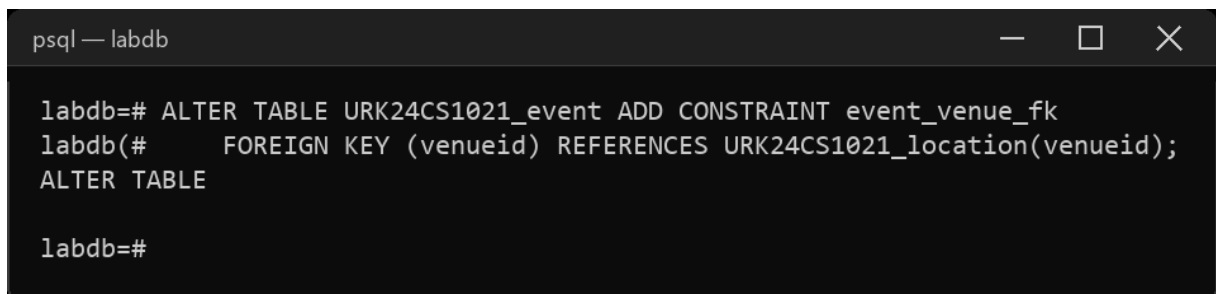
```
psql — labdb

labdb=# ALTER TABLE URK24CS1021_location ALTER COLUMN name TYPE varchar(300);
ALTER TABLE

labdb=#
```

12. Add a foreign key constraint on the VenueID column in the Event table, referencing the Venue table.

```
ALTER TABLE URK24CS1021_event ADD CONSTRAINT event_venue_fk
FOREIGN KEY (venueid) REFERENCES URK24CS1021_location(venueid);
```

A terminal window titled 'psql — labdb' with standard window controls. It shows the execution of the SQL command 'ALTER TABLE URK24CS1021_event ADD CONSTRAINT event_venue_fk FOREIGN KEY (venueid) REFERENCES URK24CS1021_location(venueid);'. The prompt 'labdb=#' is shown before and after the command, with the command itself appearing on the line between the prompts.

```
psql — labdb

labdb=# ALTER TABLE URK24CS1021_event ADD CONSTRAINT event_venue_fk
labdb=# FOREIGN KEY (venueid) REFERENCES URK24CS1021_location(venueid);
ALTER TABLE

labdb=#
```

13. Add a CHECK constraint to check whether the UserID is between 101 and 105.

```
ALTER TABLE URK24CS1021_users ADD CONSTRAINT users_id_chk CHECK (id BETWEEN 101 AND 105);
```

psql — labdb

```
labdb=# ALTER TABLE URK24CS1021_users ADD CONSTRAINT users_id_chk CHECK (id BETWEEN 101 AND 105);  
ALTER TABLE
```

```
labdb=#
```

14. Add a unique constraint to Phone column of the User table.

```
ALTER TABLE URK24CS1021_users ADD CONSTRAINT users_phone_uq UNIQUE (phone);
```

psql — labdb

```
labdb=# ALTER TABLE URK24CS1021_users ADD CONSTRAINT users_phone_uq UNIQUE (phone);  
ALTER TABLE
```

```
labdb=#
```

15. Truncate the user table.

```
TRUNCATE TABLE URK24CS1021_users;
```

psql — labdb

```
labdb=# TRUNCATE TABLE URK24CS1021_users;  
TRUNCATE TABLE
```

```
labdb=#
```

(b) Managing Tables using DML, DCL and TCL Commands

1. Insert a new user into the "User" table.

```
INSERT INTO URK24CS1021_users (id, name, email, password, phone)  
VALUES (103, 'Nina Patel', 'nina@mail.com', 'pass3', '9000000012');
```

psql — labdb

```
labdb=# INSERT INTO URK24CS1021_users (id, name, email, password, phone)  
labdb=# VALUES (103, 'Nina Patel', 'nina@mail.com', 'pass3', '9000000012');  
INSERT 0 1
```

```
labdb=#
```

2. Update the email address of a user with UserID 102.

```
UPDATE URK24CS1021_users SET email = 'mary.ann.new@mail.com' WHERE id = 102;
```

psql — labdb

```
labdb=# UPDATE URK24CS1021_users SET email = 'mary.ann.new@mail.com' WHERE id = 102;
UPDATE 1
```

labdb=#

3. Delete a user with the email "example@example.com".

```
INSERT INTO URK24CS1021_users (id, name, email, password, phone)
VALUES (104, 'Temp User', 'example@example.com', 'pass4', '9000000013');
DELETE FROM URK24CS1021_users WHERE email = 'example@example.com';
```

psql — labdb

```
labdb=# INSERT INTO URK24CS1021_users (id, name, email, password, phone)
labdb(# VALUES (104, 'Temp User', 'example@example.com', 'pass4', '9000000013');
INSERT 0 1
```

```
labdb=# DELETE FROM URK24CS1021_users WHERE email = 'example@example.com';
DELETE 1
```

labdb=#

4. Insert a new event into the "Event" table.

```
INSERT INTO URK24CS1021_event (eventid, name, eventdate, eventtime, venueid,
description)
VALUES (402, 'Food Festival', '2023-11-05', '2023-11-05 12:00:00', 1, 'Street food
festival');
```

psql — labdb

```
labdb=# INSERT INTO URK24CS1021_event (eventid, name, eventdate, eventtime, venueid, description)
labdb(# VALUES (402, 'Food Festival', '2023-11-05', '2023-11-05 12:00:00', 1, 'Street food festival');
INSERT 0 1
```

labdb=#

5. Update the description of an event with EventID 401.

```
UPDATE URK24CS1021_event SET description = 'Annual comic convention, now with a
cosplay contest'
WHERE eventid = 401;
```

psql — labdb

```
labdb=# UPDATE URK24CS1021_event SET description = 'Annual comic convention, now with a cosplay contest'
labdb(# WHERE eventid = 401;
UPDATE 1
```

labdb=#

6. Grant SELECT privileges on the "User" table to a user named "john".

```
CREATE ROLE john LOGIN;
GRANT SELECT ON URK24CS1021_users TO john;
```

psql — labdb

```
labdb=# CREATE ROLE john LOGIN;  
CREATE ROLE
```

```
labdb=# GRANT SELECT ON URK24CS1021_users TO john;  
GRANT
```

```
labdb=#
```

7. Revoke INSERT privileges on the "Event" table from a user named "mary".

```
CREATE ROLE mary LOGIN;  
GRANT INSERT ON URK24CS1021_event TO mary;  
REVOKE INSERT ON URK24CS1021_event FROM mary;
```

psql — labdb

```
labdb=# CREATE ROLE mary LOGIN;  
CREATE ROLE
```

```
labdb=# GRANT INSERT ON URK24CS1021_event TO mary;  
GRANT
```

```
labdb=# REVOKE INSERT ON URK24CS1021_event FROM mary;  
REVOKE
```

```
labdb=#
```

8. Create a new user with the username "jane" and grant them all privileges on the "Ticket" table.

```
CREATE ROLE jane LOGIN;  
GRANT ALL PRIVILEGES ON URK24CS1021_ticket TO jane;
```

psql — labdb

```
labdb=# CREATE ROLE jane LOGIN;  
CREATE ROLE
```

```
labdb=# GRANT ALL PRIVILEGES ON URK24CS1021_ticket TO jane;  
GRANT
```

```
labdb=#
```

9. Grant EXECUTE privileges on a stored procedure to a role named "admin".

```
CREATE ROLE admin;  
CREATE OR REPLACE PROCEDURE URK24CS1021_ticket_count() LANGUAGE sql AS $$ SELECT 1;  
$$;  
GRANT EXECUTE ON PROCEDURE URK24CS1021_ticket_count() TO admin;
```

psql — labdb

```
labdb=# CREATE ROLE admin;  
CREATE ROLE
```

```
labdb=# CREATE OR REPLACE PROCEDURE URK24CS1021_ticket_count() LANGUAGE sql AS $$ SELECT 1; $$;  
CREATE PROCEDURE
```

```
labdb=# GRANT EXECUTE ON PROCEDURE URK24CS1021_ticket_count() TO admin;  
GRANT
```

```
labdb=#
```

10. Revoke all privileges from a user named "guest" on all tables in the schema.

```
CREATE ROLE guest LOGIN;
```

```
REVOKE ALL PRIVILEGES ON ALL TABLES IN SCHEMA public FROM guest;
```

psql — labdb

```
labdb=# CREATE ROLE guest LOGIN;  
CREATE ROLE
```

```
labdb=# REVOKE ALL PRIVILEGES ON ALL TABLES IN SCHEMA public FROM guest;  
REVOKE
```

```
labdb=#
```

11. Perform commit a transaction in the database.

```
BEGIN;
```

```
INSERT INTO URK24CS1021_ticket (ticketid, eventid, userid, price, status, barcode)  
VALUES (902, 401, 102, 80.00, 'confirmed', 'BCX002');
```

```
COMMIT;
```

psql — labdb

```
labdb=# BEGIN;  
labdb(# INSERT INTO URK24CS1021_ticket (ticketid, eventid, userid, price, status, barcode)  
labdb(# VALUES (902, 401, 102, 80.00, 'confirmed', 'BCX002');  
labdb(# COMMIT;
```

```
BEGIN
```

```
INSERT 0 1
```

```
COMMIT
```

```
labdb=#
```

12. Perform roll back a transaction to a specific savepoint.

```
BEGIN;
```

```
INSERT INTO URK24CS1021_ticket (ticketid, eventid, userid, price, status, barcode)  
VALUES (903, 401, 103, 95.00, 'confirmed', 'BCX003');
```

```
SAVEPOINT sp1;
```

```
INSERT INTO URK24CS1021_ticket (ticketid, eventid, userid, price, status, barcode)  
VALUES (904, 401, 104, 95.00, 'confirmed', 'BCX004');
```

```
ROLLBACK TO SAVEPOINT sp1;
```

```
COMMIT;
SELECT ticketid FROM URK24CS1021_ticket WHERE ticketid IN (903, 904);
```

```
psql — labdb

labdb=# BEGIN;
labdb(# INSERT INTO URK24CS1021_ticket (ticketid, eventid, userid, price, status, barcode)
labdb(# VALUES (903, 401, 103, 95.00, 'confirmed', 'BCX003'));
labdb(# SAVEPOINT sp1;
labdb(# INSERT INTO URK24CS1021_ticket (ticketid, eventid, userid, price, status, barcode)
labdb(# VALUES (904, 401, 104, 95.00, 'confirmed', 'BCX004'));
labdb(# ROLLBACK TO SAVEPOINT sp1;
labdb(# COMMIT;
labdb(# SELECT ticketid FROM URK24CS1021_ticket WHERE ticketid IN (903, 904);
BEGIN
INSERT 0 1
SAVEPOINT
INSERT 0 1
ROLLBACK
COMMIT
 ticketid
-----
      903
(1 row)

labdb=#
```

13. Perform set a savepoint within a transaction.

```
BEGIN;
INSERT INTO URK24CS1021_ticket (ticketid, eventid, userid, price, status, barcode)
VALUES (905, 401, 105, 70.00, 'confirmed', 'BCX005');
SAVEPOINT sp2;
SELECT COUNT(*) AS tickets_so_far FROM URK24CS1021_ticket;
COMMIT;
```

```
psql — labdb

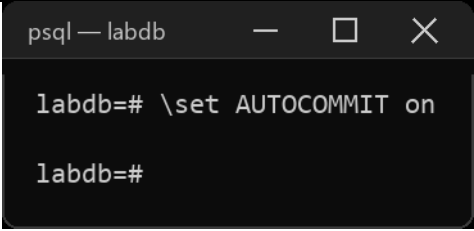
labdb=# BEGIN;
labdb(# INSERT INTO URK24CS1021_ticket (ticketid, eventid, userid, price, status, barcode)
labdb(# VALUES (905, 401, 105, 70.00, 'confirmed', 'BCX005');
labdb(# SAVEPOINT sp2;
labdb(# SELECT COUNT(*) AS tickets_so_far FROM URK24CS1021_ticket;
labdb(# COMMIT;
BEGIN
INSERT 0 1
SAVEPOINT
 tickets_so_far
-----
              4
(1 row)

COMMIT

labdb=#
```

14. Enable autocommit mode in the database.

```
\set AUTOCOMMIT on
```

A terminal window titled "psql — labdb" with standard window controls. It shows the command "\set AUTOCOMMIT on" being entered and executed, with the prompt "labdb=#" appearing twice.

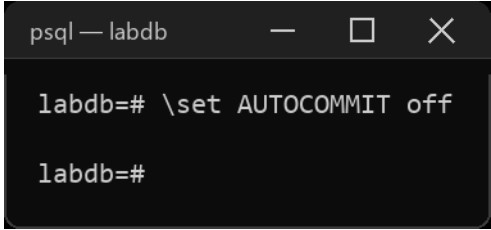
```
psql — labdb
```

```
labdb=# \set AUTOCOMMIT on
```

```
labdb=#
```

15. Disable autocommit mode in the database.

```
\set AUTOCOMMIT off
```

A terminal window titled "psql — labdb" with standard window controls. It shows the command "\set AUTOCOMMIT off" being entered and executed, with the prompt "labdb=#" appearing twice.

```
psql — labdb
```

```
labdb=# \set AUTOCOMMIT off
```

```
labdb=#
```

Result

The queries were executed and the desired output was obtained.